TRANSIX

Algoritmusok

Készítették:
Utasi Zalán Zoltán 13.I
Pálmai János Gábor 13.I

Tartalomjegyzék

1. Regisztráció (Register)
2. Bejelentkezés (Login)
3. Járatkeresés és menetrendböngészés
4. Útvonalhossz-meghatározás (GPS)
5. Viteldíj-számítás és legkedvezőbb jegy-/bérlettípus meghatározása
6. Jegyvásárlás
7. Bérletvásárlás
8. QR-kód generálás és mentés
9. Helyfoglalás (foglalás, módosítás, lemondás)
10. QR-érvényesítés (járművezetői/ellenőri alkalmazás)
11. Valós idejű járatkövetés
12. E-mail értesítések
13. Telítettségbecslés
14. Statisztikák és diszpécseri dashboard

1. Regisztráció (Register)

Cél: Új utas felhasználó létrehozása a rendszerben.

UI interakció:

1. A felhasználó a regisztrációs oldalon látja a mezőket: név, email, jelszó, jelszó megerősítése, valamint a Regisztráció gombot.
2. Kitölti a mezőket, majd rákattint a gombra.
3. Hibák esetén:
   • Üres vagy hibás formátumú mező → a mező mellett piros hibaüzenet
   • Túl rövid jelszó vagy a két jelszó nem egyezik → hibaüzenet a jelszó mező mellett
   • Már létező email → „Ez az email már foglalt”
4. Sikeres regisztráció esetén megjelenik a „Regisztráció sikeres!” üzenet.
5. A felhasználót átirányítjuk a bejelentkezési oldalra.

Bemenet: name, email, password, passwordConfirm

Kimenet: user objektum (id, name, email, role), message

UI kimenet: sikerüzenet + átirányítás a loginra, vagy hibaüzenetek

Eljárás:

1. A rendszer ellenőrzi, hogy a kérés tartalmazza a kötelező mezőket (`name`, `email`, `password`, `passwordConfirm`).
2. Ha bármelyik hiányzik → hibaüzenet visszaadása (400).
3. Ellenőrzi az email formátumát és a jelszó minimális hosszát/erősségét.
4. Ellenőrzi, hogy a `password` és a `passwordConfirm` megegyezik.
5. Megvizsgálja az adatbázisban (`users` tábla), hogy az `email` már foglalt-e.
6. Ha foglalt → hibaüzenet visszaadása (409).
7. Ha minden valid:
   a) A jelszót hash-eli (pl. BCrypt).
   b) Létrehoz egy új `User` rekordot, a `role` értéke alapértelmezetten `passenger`.
   c) Elmenti a rekordot az adatbázisba.
8. Visszaadja az új felhasználó adatait (ID, név, email, role) JSON formátumban, a jelszó hash nélkül (201).

Megjegyzés: A járművezető, diszpécser és admin szerepköröket csak adminisztrátor adhatja ki, ezért a regisztráció mindig utas szerepkört hoz létre.

2. Bejelentkezés (Login)

Cél: A felhasználó azonosítása és JWT hozzáférési token generálása.

UI interakció:

1. A felhasználó a bejelentkezési oldalon látja az email és jelszó mezőket, valamint a Bejelentkezés gombot.
2. Kitölti a mezőket, majd rákattint a gombra.
3. Hiba esetén a mezők felett hibaüzenet jelenik meg:
   - Üres mező → „Az email és a jelszó megadása kötelező”
   - Rossz adatok → „Érvénytelen email vagy jelszó”
   - Letiltott fiók → „A fiók le van tiltva”
4. Sikeres bejelentkezés esetén a rendszer a szerepkör alapján átirányít:
   - utas → főoldal / saját jegyek, bérletek
   - járművezető/ellenőr → érvényesítő felület
   - diszpécser, admin → dashboard
5. A token tárolása a kliensen: weben localStorage vagy HttpOnly cookie, mobilon biztonságos tároló (Flutter Secure Storage).
6. A token a további API-hívásoknál az `Authorization: Bearer <token>` fejlécben megy.

Bemenet: email, password

Kimenet: access_token, token_type, expires_in, user objektum (id, name, email, role)

UI kimenet: hibaüzenet, vagy átirányítás a szerepkörnek megfelelő felületre

Eljárás:

1. A rendszer ellenőrzi, hogy a kérés tartalmazza az `email` és `password` mezőket.
2. Ha bármelyik hiányzik → hibaüzenet (400).
3. Kikeresi az adatbázisból (`users` tábla) az adott `email`-hez tartozó felhasználót.
4. Ha nincs ilyen user → hibás hitelesítés (401). Ugyanazt az üzenetet adja, mint rossz jelszónál.
5. Ellenőrzi a jelszó egyezését a tárolt hash (`passwordHash`) és a beérkező jelszó között.
6. Ha nem egyezik → hibás hitelesítés (401).
7. Ellenőrzi, hogy a fiók aktív-e. Ha le van tiltva → hibaüzenet (403).
8. Ha minden rendben van:
   a) Összeállítja a JWT claim-eket: `sub` (user ID), `email`, `role`, `exp` (lejárat, pl. 60 perc).
   b) Aláírja a tokent a szerver titkos kulcsával (HMAC SHA-256).
   c) Opcionálisan refresh tokent generál és elmenti az adatbázisba.
9. Visszaadja a tokent és a felhasználó azonosító adatait JSON formátumban (200).

Megjegyzés: A titkos kulcs ne kerüljön GitHubra, környezeti változóból vagy user-secrets-ből jöjjön.

3. Járatkeresés és menetrendböngészés

Cél: A felhasználó megtalálja a számára megfelelő járatokat indulási és érkezési megálló, dátum és további szűrők alapján.

UI interakció:

1. A főoldalon (és a járatkereső oldalon) a felhasználó látja a gyorskereső felületet: honnan, hová, dátum, utasok száma.
2. Megadja a megállókat (automatikus kiegészítéssel) és a dátumot, majd a Keresés gombra kattint.
3. Az eredménylista rendezhető (indulás, ár, utazási idő) és szűrhető (időszak, telítettség, járattípus).
4. Egy járatra kattintva megjelennek a részletek: megállók, érkezési idők, ár, telítettségjelző.
5. Nincs találat esetén: „Nincs a keresésnek megfelelő járat”, javaslattal a szomszédos napokra.
6. Érvénytelen adat (múltbeli dátum, azonos indulási és érkezési megálló) → hibaüzenet.

Bemenet: fromStopId, toStopId, date, passengers, filters (időszak, rendezés, oldalszám)

Kimenet: járatok listája (tripId, indulás, érkezés, időtartam, ár, szabad helyek, telítettségi szint)

UI kimenet: találati lista vagy hibaüzenet / üres lista üzenet

Eljárás:

1. Ellenőrzi a kötelező paramétereket és azok érvényességét (dátum nem múltbeli, a két megálló különbözik). Hiba esetén 400.
2. Megkeresi a `route_stops` táblában azokat az útvonalakat, amelyek érintik az indulási majd az érkezési megállót ebben a sorrendben.
3. Lekérdezi a `trips` táblából az ilyen útvonalak adott napi, nem törölt járatait.
4. Kiszámolja minden járatra az indulási és érkezési időt a megállóknál (menetrendi idő + késés).
5. Meghívja az útvonalhossz- és díjszámító modult a járat árához (4–5. algoritmus).
6. Meghívja a telítettségbecslőt a járat szintjéhez (13. algoritmus).
7. Alkalmazza a szűrőket és a rendezést, majd lapozza az eredményt.
8. Visszaadja a találati listát JSON formátumban (200).

4. Útvonalhossz-meghatározás (GPS)

Cél: A megtett vagy tervezett útszakasz hosszának automatikus meghatározása GPS-koordináták és megállók alapján.

UI interakció:

1. Az utas jegyvásárláskor kiválasztja a fel- és leszállási megállót, vagy a térképen jelöli azokat.
2. A rendszer megjeleníti a számított távolságot (km) a kiválasztás mellett.
3. Hiba esetén (a leszállási megálló az útvonalon az indulás előtt van, hiányzó koordináta) hibaüzenet jelenik meg.

Bemenet: tripId (vagy routeId), boardingStopId, alightingStopId

Kimenet: distanceKm (két tizedesre kerekítve)

UI kimenet: távolság megjelenítése, vagy hibaüzenet

Eljárás:

1. Betölti az útvonal megállóit sorrendben a `route_stops` táblából (koordináták: `latitude`, `longitude`).
2. Ellenőrzi, hogy mindkét megálló szerepel az útvonalon, és a felszállás sorszáma kisebb a leszállásénál. Különben hiba (400).
3. Ha a `route_stops` tábla tartalmaz előre számolt kumulált távolságot (`cumulativeKm`), a távolság a leszállási és a felszállási érték különbsége.
4. Egyébként a két megálló közötti minden szomszédos megállópárra kiszámolja a távolságot a Haversine-képlettel:
   a) $a = \sin^2(\Delta\varphi/2) + \cos\varphi1 \cdot \cos\varphi2 \cdot \sin^2(\Delta\lambda/2)$
   b) $d = 2 \cdot R \cdot \text{atan2}(\sqrt{a}, \sqrt{(1-a)})$, ahol $R = 6371$ km
5. Összegzi a szakaszok hosszát.
6. Az útvonal tényleges, úthálózat szerinti hosszához korrekciós tényezőt alkalmaz (pl. 1,2), ha csak légvonalbeli adat áll rendelkezésre.
7. A járat lefutása után a rögzített GPS-pozíciókból (`vehicle_positions`) is kiszámolható a ténylegesen megtett táv, ez ellenőrzésre vagy statisztikára használható.
8. Visszaadja a távolságot km-ben.

5. Viteldíj-számítás és legkedvezőbb jegy-/bérlettípus meghatározása

Cél: Az utazás távolsága alapján a viteldíj kiszámítása, valamint a felhasználó utazási szokásai alapján a legkedvezőbb jegy- vagy bérlettípus (nap, hét, hónap, év) kiválasztása.

UI interakció:

1. Jegyvásárláskor a felhasználó látja az egyszeri jegy árát.
2. Ha az utazási előzményei alapján bérlet olcsóbb lenne, a rendszer ajánlatot jelenít meg: „Havi bérlettel X Ft-ot spórolhat”.
3. A felhasználó választhat az ajánlott és az egyszeri jegy között.
4. Ha nincs elég előzmény, csak az egyszeri jegy ára jelenik meg.

Bemenet: userId, distanceKm, travelDate, kedvezménykategória (pl. diák, nyugdíjas)

Kimenet: singleFare, ajánlott típus (single / daily / weekly / monthly / yearly), ár, becsült megtakarítás

UI kimenet: ár és ajánlat megjelenítése

Eljárás:

1. Lekéri a `tariffs` táblából az aktív díjszabást (alapdíj, kilométerdíj, sávhatárok, kedvezmények).
2. Kiszámolja az egyszeri jegy árát: `ár = alapdíj + km × kilométerdíj` (a megfelelő távolsági sávban), majd levonja az érvényes kedvezményt.
3. Az árat a tarifában megadott módon kerekíti (pl. 5 Ft-ra).
4. Ha a felhasználó be van jelentkezve, lekéri az utazási előzményeit (`tickets`, `validations`) az elmúlt időszakra (pl. 30 nap).
5. Az előzményekből megbecsli a várható utazások számát és összköltségét a lehetséges időszakokra (nap, hét, hónap, év).
6. Összehasonlítja a becsült összköltséget az adott időszakra szóló bérlet árával (`pass_types` tábla).
7. Kiválasztja azt a lehetőséget, amelynek a becsült teljes költsége a legalacsonyabb.
8. Kiszámolja a megtakarítást az egyszeri jegyekhez képest. Ha nem pozitív, az egyszeri jegyet ajánlja.
9. Visszaadja az árat, az ajánlott típust és a megtakarítást JSON formátumban.

Megjegyzés: Az ajánlás becslés, a felhasználó mindig szabadon választhat.

6. Jegyvásárlás

Cél: Egyszeri jegy megvásárlása egy kiválasztott járatra és szakaszra.

UI interakció:

1. A felhasználó a járat részleteinél a Jegyvásárlás gombra kattint.
2. Kiválasztja a fel- és leszállási megállót, a jegytípust (teljes árú, kedvezményes) és szükség esetén az ülőhelyet.
3. Látja a végösszeget, a telítettségjelzőt és az esetleges bérletajánlatot.
4. Megadja a fizetési adatokat és a Fizetés gombra kattint.
5. Hibák: elfogyott a hely, a járat már elindult, sikertelen fizetés → hibaüzenet.
6. Sikeres vásárlás után megjelenik a „Sikeres vásárlás!” üzenet és a jegy QR-kódja, majd átirányítás a saját jegyek oldalra.

Bemenet: userId, tripId, boardingStopId, alightingStopId, ticketType, seatId (opcionális), paymentData

Kimenet: ticket objektum (id, QR-kód, ár, státusz), message

UI kimenet: sikerüzenet, digitális jegy QR-kóddal, vagy hibaüzenet

Eljárás:

1. Ellenőrzi a bejövő adatokat (a felhasználó, a járat, a megállók létezése, a járat még nem indult el, a jegytípus érvényes).
2. Ellenőrzi, hogy van-e szabad hely a kiválasztott szakaszon (13. algoritmus kapacitásszámítása). Ha nincs → hiba (409).
3. Kiszámolja az árat a 4–5. algoritmussal.
4. Ha ülőhelyet is választott, a 9. algoritmus szerint ideiglenesen zárolja az ülőt.
5. Feldolgozza a fizetést a fizetési szolgáltatón keresztül, és létrehoz egy `Payment` rekordot.
6. Sikertelen fizetésnél felszabadítja a zárolást, és hibát ad vissza (402).
7. Sikeres fizetésnél adatbázis-tranzakcióban:
   a) létrehoz egy `Ticket` rekordot (`status = active`),
   b) véglegesíti az ülőhelyfoglalást,
   c) meghívja a QR-generálást (8. algoritmus).
8. Visszaadja a jegy adatait JSON formátumban (201), és e-mail visszaigazolást küld.

7. Bérletvásárlás

Cél: Időszakos bérlet (napi, heti, havi, éves) megvásárlása és lejáratának rögzítése.

UI interakció:

1. A felhasználó a Bérletek oldalon látja az elérhető bérlettípusokat árakkal és érvényességgel.
2. Kiválasztja a bérlettípust, a kezdő dátumot és az érvényes útvonalat vagy zónát.
3. Megadja a fizetési adatokat, majd a Vásárlás gombra kattint.
4. Hibák: érvénytelen kezdő dátum, már létező, átfedő bérlet, sikertelen fizetés → hibaüzenet.
5. Siker esetén megjelenik a „Bérlet megvásárolva!” üzenet és a bérlet QR-kódja a saját bérletek oldalon.

Bemenet: userId, passTypeId, startDate, routeId (opcionális), paymentData

Kimenet: pass objektum (id, QR-kód, érvényesség kezdete és vége, státusz), message

UI kimenet: sikerüzenet és digitális bérlet, vagy hibaüzenet

Eljárás:

1. Ellenőrzi a bemenetet (felhasználó, bérlettípus létezik és aktív, kezdő dátum nem múltbeli).
2. Megvizsgálja, hogy a felhasználónak van-e már ugyanarra az útvonalra átfedő aktív bérlete. Ha van → hiba (409).
3. Lekéri a bérlettípus árát és az érvényes kedvezményt, kiszámolja a végösszeget.
4. Kiszámolja a lejáratot: `lejárat = kezdő dátum + időtartam` (nap/hét/hónap/év).
5. Feldolgozza a fizetést és létrehoz egy `Payment` rekordot.
6. Sikeres fizetés után adatbázis-tranzakcióban létrehoz egy `Pass` rekordot (`status = active`).
7. Meghívja a QR-generálást (8. algoritmus).
8. A lejárat előtti emlékeztetőt beütemezi (12. algoritmus).
9. Visszaadja a bérlet adatait JSON formátumban (201).

8. QR-kód generálás és mentés

Cél: A megvásárolt jegyekhez és bérletekhez egyedi azonosító és hamisítás ellen védett QR-kód létrehozása és mentése az adatbázisba.

UI interakció:

1. A felhasználó a vásárlás után a saját jegyek/bérletek oldalon látja a QR-kódot.
2. A QR-kód a mobilalkalmazásban offline is megjeleníthető, ha egyszer már letöltődött.
3. Hiba esetén (a kód nem generálható) a felhasználó értesítést kap, és újrageneráltathatja a kódot.

Bemenet: ticketId vagy passId, userId, érvényességi adatok

Kimenet: uniqueCode, qrPayload, qrImage (Base64 vagy URL)

UI kimenet: a QR-kód képe a digitális jegyen/bérleten

Eljárás:

1. Generál egy egyedi azonosítót (UUID), amely a jegy vagy bérlet elsődleges kulcsától független.
2. Összeállítja a QR-kód tartalmát (payload): típus (`ticket`/`pass`), azonosító, felhasználó, érvényesség vége.
3. Digitális aláírással (HMAC SHA-256, szerveroldali titkos kulccsal) ellátja a payloadot, hogy a kliens ne tudja meghamisítani.
4. A payload és az aláírás alapján QR-kód képet készít (pl. QRCoder könyvtárral).
5. Elmenti az adatbázisba az egyedi azonosítót és az aláírást (a `tickets`/`passes` táblában).
6. Ellenőrzi, hogy az egyedi azonosító valóban egyedi (ütközés esetén újragenerál).
7. Visszaadja a QR-kódot a hívónak.

Megjegyzés: A QR-kód ne tartalmazzon személyes adatot, csak azonosítót és aláírást.

9. Helyfoglalás (foglalás, módosítás, lemondás)

Cél: Az utas a jármű alaprajzán kiválaszthatja, módosíthatja vagy lemondhatja az ülőhelyét az indulás előtt.

UI interakció:

1. A felhasználó a jegy részleteinél megnyitja a Helyfoglalás nézetet, ahol a jármű alaprajza látható.
2. A szabad, foglalt és saját helyek különböző színnel jelennek meg.
3. Rákattint egy szabad ülőhelyre, majd a Foglalás gombra. Módosításnál másik helyet választ, lemondásnál a Lemondás gombra kattint.
4. Hibák: a hely közben foglalttá vált, a járat már elindult → üzenet és az alaprajz frissítése.
5. Siker esetén megjelenik a „Foglalás sikeres!” üzenet, és az új hely a jegyen is látszik.

Bemenet: userId, ticketId, tripId, seatId, művelet (create / change / cancel)

Kimenet: seatReservation objektum (id, seatId, status), message

UI kimenet: frissített alaprajz, sikerüzenet vagy hibaüzenet

Eljárás:

1. Lekéri a jármű alaprajzát (`vehicles`, `seats`) és a járat foglalt helyeit a `seat_reservations` táblából a szakaszra vonatkozóan.
2. Ellenőrzi, hogy a jegy a felhasználóé és érvényes, a járat még nem indult el (a módosítási határidő a beállításokban megadható).
3. Foglalás esetén:
   a) ellenőrzi, hogy a kiválasztott hely szabad a teljes szakaszon,
   b) tranzakcióban létrehoz egy `SeatReservation` rekordot (`status = reserved`); az egyediségi kulcs (járat, ülés, szakasz) védi a dupla foglalástól.
4. Módosítás esetén tranzakcióban a régi foglalást felszabadítja (`status = released`), és az újat létrehozza. Ha az új hely foglalt, a régi marad.
5. Lemondás esetén a foglalás `status` mezője `cancelled` lesz, a hely újra szabad.
6. Frissíti a járat telítettségi adatait (13. algoritmus).
7. Visszaadja a foglalás adatait JSON formátumban (200/201).

10. QR-érvényesítés (járművezetői/ellenőri alkalmazás)

Cél: A járművezető vagy ellenőr gyorsan és megbízhatóan ellenőrzi az utasok jegyét vagy bérletét a QR-kód beolvasásával.

UI interakció:

1. A járművezető a nagy elemekkel kialakított érvényesítő képernyőn látja a kamera képét és az aktuális járat nevét.
2. A kamera beolvassa az utas QR-kódját.
3. Az eredmény egyértelmű, nagy méretű visszajelzésként jelenik meg: zöld (érvényes), piros (érvénytelen, lejárt, már felhasznált, más járat).
4. A hiba okát szöveg is jelzi, és hang- vagy rezgésjelzés kíséri.
5. A beolvasott utasok az utaslistában is megjelennek.

Bemenet: qrPayload (+ aláírás), tripId, driverId, boardingStopId

Kimenet: validationResult (valid / invalid), ok, jegy/bérlet alapadatai, validation rekord

UI kimenet: zöld vagy piros eredménykijelzés és az utaslista frissítése

Eljárás:

1. Dekódolja a QR-kód tartalmát, és ellenőrzi az aláírást a szerver titkos kulcsával. Hibás aláírás → érvénytelen (hamis kód).
2. Kikeresi az adatbázisból a jegyet vagy bérletet az azonosító alapján. Ha nem található → érvénytelen.
3. Ellenőrzi az állapotot: lemondott, visszatérített vagy letiltott → érvénytelen.
4. Jegy esetén ellenőrzi, hogy a jegy az adott járatra és szakaszra szól, és a felszállási megálló megfelel. Más járat → érvénytelen.
5. Jegy esetén ellenőrzi, hogy még nem lett beváltva (`status = active`). Már felhasznált → érvénytelen (duplikáció).
6. Bérlet esetén ellenőrzi, hogy az érvényességi időn belül van, és az útvonalra érvényes.
7. Érvényes jegyet beváltottra állít (`status = used`), a bérletnél nem változtat az állapoton.
8. Létrehoz egy `Validation` naplórekordot (idő, járat, járművezető, eredmény, ok).
9. Visszaadja az eredményt JSON formátumban (200).
10. Hálózat nélkül az alkalmazás helyben tárolja az érvényesítéseket, és visszatéréskor szinkronizálja.

11. Valós idejű járatkövetés

Cél: A járművek aktuális helyzetének rögzítése és megjelenítése az utasoknak és a diszpécsernek.

UI interakció:

1. Az utas a mobilalkalmazásban a Járatkövetés képernyőn térképen látja a járatot és a várható érkezési időt.
2. A jármű ikonja néhány másodpercenként frissül.
3. Késés esetén a rendszer jelzi a késés mértékét, a diszpécser a dashboardon az összes aktív járatot látja.
4. Ha a jármű pozíciója nem frissül (pl. jelvesztés), az alkalmazás jelzi, hogy az adat nem friss.

Bemenet: (adóoldal) vehicleId/tripId, latitude, longitude, speed, timestamp; (olvasóoldal) tripId

Kimenet: aktuális pozíció, következő megálló, becsült érkezési idő, késés percben

UI kimenet: jármű mozgása a térképen, ETA és késés kijelzése

Eljárás:

1. A járművezetői alkalmazás meghatározott időközönként (pl. 5–10 mp) elküldi a GPS-pozíciót az API-nak.
2. A szerver ellenőrzi a hitelesítést és azt, hogy a járművezető ahhoz a járathoz van rendelve, amelyre a pozíciót küldi.
3. Validálja az adatokat (koordináták értéktartománya, értelmes időbélyeg). Hibás adat elvetve.
4. Elmenti a pozíciót a `vehicle_positions` táblába, és az utolsó ismert pozíciót frissíti.
5. Meghatározza a legközelebbi megállót, és észleli a megállóhoz érkezést (adott sugarú körbe lépés, pl. 100 m).
6. Kiszámolja a késést: tényleges érkezés és menetrendi idő különbsége; a hátralévő megállókra becsült érkezési időt.
7. Ha a késés meghalad egy küszöböt (pl. 10 perc), értesítést indít (12. algoritmus).
8. Az utasoknak és a diszpécsernek a friss pozíciót valós időben (SignalR/WebSocket) küldi, vagy az ügyfél rövid időközönként lekérdezi (polling).

12. E-mail értesítések

Cél: Az utasok időben értesülnek a járat indulásáról, késéséről, útvonalváltozásáról, valamint a jegy vagy bérlet lejáratáról.

UI interakció:

1. A felhasználó a Beállítások oldalon kapcsolhatja az értesítéstípusokat (indulás előtt, késés, útvonalváltozás, lejárat).
2. Az értesítés e-mailben érkezik a regisztrált címre, a vonatkozó járat vagy jegy adataival.
3. Az értesítésben link vezet a jegy vagy járat részleteihez.

Bemenet: esemény típusa (reminder / delay / routeChange / expiry), tripId vagy ticketId/passId

Kimenet: elküldött e-mailek, `notifications` napló

UI kimenet: e-mail az utasnak, a küldés ténye a naplóban

Eljárás:

1. Az eseményt kiváltja egy háttérfeladat (időzítő) vagy egy másik modul (késés észlelése, útvonal módosítása a diszpécserek által).
2. Időzített feladat (pl. percenként): megkeresi a közelgő járatokat (pl. 60 perc múlva indul) és a hamarosan lejáró jegyeket/bérleteket (pl. 3 nap múlva).
3. Kikeresi az érintett utasokat (érvényes jeggyel, foglalással a járatra, vagy a lejáró bérlet tulajdonosa).
4. Ellenőrzi az utas értesítési beállításait és az e-mail címet. Ha az adott típust kikapcsolta, kihagyja.
5. Ellenőrzi a `notifications` táblában, hogy ugyanaz az értesítés nem ment-e már ki (duplikáció elkerülése).
6. Összeállítja az e-mail tartalmát sablon alapján (járat, idők, ok, teendő).
7. Elküldi az e-mailt SMTP-n keresztül.
8. Elmenti a küldés eredményét (siker/hiba) a `notifications` táblába, hiba esetén később újrapróbálja (pl. 3-szor).

13. Telítettségbecslés

Cél: Az eladott, beváltott jegyek és a helyfoglalások alapján becslés adása a járat várható zsúfoltságáról.

UI interakció:

1. Jegyvásárláskor és a járatlistában az utas színes jelzést lát: zöld (kevés utas), sárga (közepes), piros (zsúfolt).
2. A diszpécser a dashboardon járatonként látja a telítettséget százalékban és a szakaszok szerinti bontást.
3. Telt házas járatnál a vásárlás gomb letiltott, a felület másik járatot javasol.

Bemenet: tripId, (opcionális) boardingStopId, alightingStopId

Kimenet: occupancyPercent, szint (low / medium / high / full), szabad helyek száma

UI kimenet: színjelzés és szöveg az utasnak, részletes nézet a diszpécsernek

Eljárás:

1. Lekéri a járat kapacitását a járműből (`vehicles.capacity`).
2. Megszámolja az aktív jegyeket és a foglalásokat (`tickets`, `seat_reservations`) a járatra, szakaszonként.
3. Egy szakaszon a foglaltság: az adott szakaszt érintő érvényes jegyek és foglalások száma. A járat telítettsége a szakaszok maximuma.
4. Már elindult járatnál a beváltott jegyek és a tényleges fel- és leszállások (`validations`) alapján pontosít.
5. Kiszámolja a telítettséget: `foglaltság % = foglalt helyek / kapacitás × 100`.
6. Szintbe sorolja az értéket (pl. alacsony < 50%, közepes 50–80%, magas 80–100%, tele = 100%). A határok a beállításokban módosíthatók.
7. Opcionálisan korrigálja a becslést a korábbi, azonos napszakú járatok átlagos beváltási arányával (nem minden eladott jegyet váltanak be).
8. Visszaadja az eredményt JSON formátumban.

14. Statisztikák és diszpécseri dashboard

Cél: Az eladási és beléptetési adatokból statisztikák, rendezhető táblázatok és diagramok előállítása a diszpécser és az admin számára.

UI interakció:

1. A diszpécser/admin a dashboardon látja a fő mutatókat: napi bevétel, eladott jegyek, aktív járatok, késések, telítettség.
2. Szűrőket állíthat be: időszak, útvonal, járat, jegytípus, felhasználói csoport.
3. Az adatok rendezhető táblázatban és diagramokon (vonal-, oszlop-, kördiagram) jelennek meg.
4. Az adatok exportálhatók (CSV/PDF).
5. Jogosulatlan felhasználónak hibaüzenet jelenik meg (nincs hozzáférés).

Bemenet: szűrők (dateFrom, dateTo, routeId, tripId, ticketType, groupBy, sortBy, sortDir, page)

Kimenet: összesített mutatók, táblázatos adatsorok, diagramadatok

UI kimenet: dashboard kártyák, táblázat és diagramok

Eljárás:

1. Ellenőrzi a JWT tokent és a szerepkört (diszpécser vagy admin). Egyébként 403.
2. Validálja a szűrőket (dátumtartomány, értelmes rendezési mező).
3. Összesítő lekérdezéseket futtat az adatbázison (`tickets`, `passes`, `payments`, `validations`):
   a) bevétel és eladott jegyek száma az időszakban (csoportosítva nap/hét/hónap szerint),
   b) beváltott jegyek és bérletek száma járatonként, útvonalanként,
   c) legnépszerűbb útvonalak és időpontok, átlagos telítettség,
   d) késések átlaga, törölt járatok száma.
4. Alkalmazza a szűrést, rendezést és lapozást a táblázatos nézethez.
5. A diagramokhoz idősoros vagy kategóriák szerinti adatsorokat készít.
6. Visszaadja az eredményt JSON formátumban (200). Export esetén CSV/PDF fájlt generál.
7. A gyakran használt lekérdezéseket gyorsítótárazza, hogy a dashboard gyors maradjon.